Lesson 9 Teacher's Key: On the Scoreboard

GameMaker - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

That GameMaker's Draw GUI event has been the 'real' HUD tool the whole time, not a placeholder - this lesson simply asks it to draw two more things, every frame, the same way it's already drawing conditional win/lose messages.

Level 1 - Full Worked Solution

- __dnd_score is a new instance variable on obj_gamemanager, parallel to enemies_remaining and __dnd_lives - the same variable GameMaker's drag-and-drop Score blocks use, so Code and Visual students are working with identical data.
- The score increment lives in obj_bullet's Collision event - the exact same place enemies_remaining already changes, since both are consequences of the exact same confirmed hit.
- The two new draw_text() calls are unconditional (no if-check around them, unlike has_won/has_lost) - they simply run every single frame, which is exactly what a persistent HUD needs.

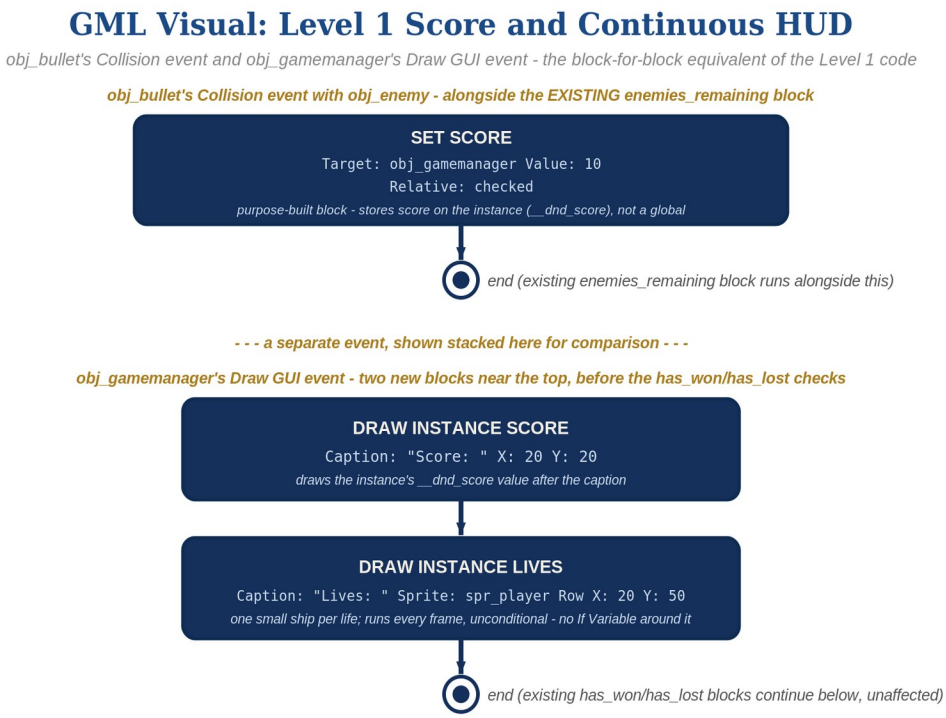
Highlighted lines are new this week; everything else is code you already have.

```
// obj_gamemanager's Create event:
enemies_remaining = 5;
has_won = false;
__dnd_lives = 3;
has_lost = false;
__dnd_score = 0;

// obj_bullet's Collision event with obj_enemy:
audio_play_sound(sfx_hit, 1, false);
obj_gamemanager.enemies_remaining -= 1;
obj_gamemanager.__dnd_score += 10;
if (obj_gamemanager.enemies_remaining <= 0) {
    obj_gamemanager.has_won = true;
}
instance_destroy(other);
instance_destroy(self);

// obj_gamemanager's Draw GUI event:
draw_text(20, 20, "Score: " + string(__dnd_score));
draw_text(20, 50, "Lives: " + string(__dnd_lives));
if (has_won) {
```

```
    draw_text(400, 50, "You Win!");
}
if (has_lost) {
    draw_text(400, 90, "Game Over");
}
```



obj_bullet's Collision event and obj_gamemanager's Draw GUI event - the Visual (drag-and-drop) equivalent of this solution, for students comparing both paths.

Why This Structure

- Draw GUI was already the correct, idiomatic GameMaker tool for HUD elements as far back as Lesson 6's win message - this lesson doesn't introduce a new mechanism, it just uses the existing one unconditionally instead of conditionally.
- Because Draw GUI re-reads __dnd_score and __dnd_lives directly from the variables every frame (no separate cached copy), there's no possible way for the display to fall out of sync - a nice, simple guarantee worth naming explicitly.
- Visual path: purpose-built blocks throughout - Set Score (Relative, Value 10, Target obj_gamemanager) in obj_bullet's Collision event, then Draw Instance Score and Draw Instance Lives (spr_player) in Draw GUI. Because the code path uses the same __dnd_score/__dnd_lives variables, a student can mix typed code and blocks in one project. Draw Instance Lives only draws frame 0 of its sprite and ignores image_xscale/image_blend - fine for spr_player.

Level 2 / Level 3 Talking Points

- Ask students to contrast this week's unconditional draw_text() calls with the existing conditional ones (has_won/has_lost) - same event, same function, different condition logic. A good moment to reinforce that "always show" and "show only when X" are both just conditions (or the absence of one).
- For students curious about visual polish: draw_set_font() and draw_set_color() are worth a quick live demo, since GameMaker's default draw_text() styling is quite plain.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: 3 kills read exactly "Score: 30"; an escape moves lives but
never score;
// both numbers are correct the instant the game starts, not only after
the first kill or escape.
```

Level 3 - complete code (all goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// obj_bullet's Collision event with obj_enemy - Level 3:
audio_play_sound(sfx_hit, 1, false);
obj_gamemanager.enemies_remaining -= 1;
obj_gamemanager.__dnd_score += irandom_range(5, 15);
if (obj_gamemanager.enemies_remaining <= 0) {
    obj_gamemanager.has_won = true;
}
instance_destroy(other);
instance_destroy(self);

// obj_gamemanager's Draw GUI event - Level 3 styling:
draw_set_color(c_yellow);
draw_text(20, 20, "Score: " + string(__dnd_score));
draw_text(20, 50, "Lives: " + string(__dnd_lives));
draw_set_color(c_white);
if (has_won) {
    draw_text(400, 50, "You Win!");
}
if (has_lost) {
    draw_text(400, 90, "Game Over");
}

// Visual path: Set Score with Value 10 becomes Value irandom_range(5, 15)
(Expression); a Set Color Draw block before the Draw Instance Score block
gives the styling.
```

Common Student Mistakes

- Adding the new draw_text() calls to the regular Draw event instead of Draw GUI (causes the HUD to scroll/zoom with the room camera instead of staying fixed on screen).
- Adding the score increment to obj_enemy's Step event by mistake instead of obj_bullet's Collision event (makes escaping award points, which is backwards).
- Y-coordinate collisions between the new score/lives text and the existing has_won/has_lost messages, especially once both are visible at once near end-of-game.